

Evolution Report

Workflow: "Gemini vs Groq Evaluation Loop (Fixed)" | n8n Automation Pipeline

1. What the Workflow Does

This is an n8n pipeline that benchmarks two LLM providers — Google Gemini 1.5 Flash and Groq — on an email classification task. It takes 10 hardcoded test emails (each with a pre-labeled expected category: Sales, Support, or Complaint), runs each one through both a Gemini chain and a Groq chain in parallel, scores the output as pass/fail against the expected label, estimates per-call cost and latency, merges the two branches back together, waits 15 seconds between iterations to respect rate limits, loops to the next test item, and finally aggregates everything into a summary report that recommends a winning provider.

2. Evidence of a Prior Version → Current Version

The two scoring nodes (**Calculate Gemini Latency** and **Calculate Groq Latency**) contain inline code comments that document exactly what changed from an earlier iteration:

- **Real cost estimation replaces a flat constant.** The previous version likely charged a fixed cost per call regardless of prompt or response size. The current version derives cost from character-count-based token estimates (~4 characters per token), with a fallback to real tokenUsage / usage_metadata fields when the model node exposes them.
- **Graceful fallback logic.** Optional chaining (usage?.promptTokens ?? ... ?? Math.ceil(...)) means a missing usage object no longer breaks the cost calculation.
- **Errors are now explicitly marked instead of silently scored as FAIL.** Failed LLM calls are tagged with gemini_status: "ERROR" / groq_status: "ERROR", with the raw error captured separately in gemini_error / groq_error. Previously an empty output from a failed call would have been indistinguishable from a genuine wrong-answer FAIL.

This is a meaningful, well-targeted fix: it moves the evaluation from a "black box pass rate" toward something closer to a genuine benchmarking harness — separating infrastructure failures from model failures, and providing directional cost signal instead of a fabricated constant.

3. Structural Review of the Current Version

Strengths

- Parallel branch design (Gemini and Groq run per item, merged by position) is efficient and keeps a clean 1:1 result row per test case.
- onError: continueRegularOutput is correctly set on both LLM chain nodes, so a single failed call doesn't halt the whole loop — this is exactly what the new ERROR-tagging logic depends on.
- The Summary Report Generator cleanly aggregates accuracy %, average latency, total cost, and error counts per model, alongside a full results table for auditing.

Remaining Gaps & Risks

- **Scoring is substring match, not exact/semantic match.** `outputText.includes(expectedText)` means a model that answers "I'd classify this as Support, but it could also be Sales" would still PASS, even though it ignored the explicit "return ONLY the category name" instruction. A stricter equality check after trim/lowercase would better reflect instruction-following.
- **The final recommendation ignores cost, latency, and errors.** `recommendation: gPassed >= qPassed ? "Google Gemini" : "Groq"` compares raw pass counts only. Despite computing latency and cost for both providers, neither factors into the verdict — a 1-pass edge with 10x the latency would still "win."
- **Latency timers include Code-node overhead.** `gemini_start/groq_start` are set before the HTTP call but also capture the preceding Code node's execution time. Negligible for 10 items, but worth noting if these numbers are used for precise comparisons.
- **Cost constants are hardcoded, undated estimates.** Pricing constants are labeled "approx" in comments but the final report prints a dollar figure with no caveat that it's an estimate, and no note of which date/rate they reflect.
- **The 15-second wait is unconditional.** Fine for a 10-item test run, but would benefit from becoming conditional or backoff-based if the test set scales up.

4. Bottom Line

The changes reflected in the "(Fixed)" version are a real improvement: better cost modeling, graceful fallback handling, and correct separation of infrastructure errors from genuine model failures. The next highest-value fix would be tightening the pass/fail comparison from substring match to exact match, and folding latency, cost, and error rate into the recommendation logic — since two of the three metrics the workflow already computes currently have no bearing on its final conclusion.